

Reproducing Med-NCA: A Rigorous Reproducibility Study of Neural Cellular Automata for Medical Image Segmentation

legacccy (Yu Jia)

Department of Bioinformatics, XJTLU

Reproduction target: Med-NCA (IPMI 2023) & M3D-NCA (MICCAI 2023)

June 2026

Abstract

Neural Cellular Automata (NCA) have emerged as an extremely lightweight ($\sim 25\text{k}$ – 70k parameters) alternative to U-Nets for medical image segmentation, with Med-NCA reporting U-Net-competitive Dice on the Medical Segmentation Decathlon (MSD). We present an *independent, zero-deviation* reproduction of Med-NCA on its two original benchmarks — Hippocampus (MSD Task04) and Prostate (MSD Task05) — following the RIDGE reproducibility framework, and we report three findings. **(1)** The Hippocampus anchor reproduces faithfully: per-volume Dice = 0.864 against the paper’s 0.886, within one paper standard deviation. **(2)** The Prostate anchor is, under the official public code and a strict zero-deviation protocol, *not reproducible at paper scale*. We initially hypothesised the gap was under-training (301 vs. 1000 epochs); we falsified this with a controlled 9-seed $\times$ 1000-epoch sweep in which *0/9 runs survived* — and, counting all attempts, 0/11 official-config runs ever reached 1000 epochs. The single best partial run scores Dice = 0.672 (paper 0.838). **(3)** We trace the instability to a concrete and transferable mechanism: NCA’s stochastic *fire mask* (50% per-step cell dropout, $\times 64$ steps) draws from a CUDA RNG stream that `numpy/torch` CPU seeding does *not* control. Whether a run lands in a converging or a diverging basin is decided *at epoch 1* by this uncontrolled randomness — the *same* seed (42) produced an epoch-1 loss of 1.25 (converged) in one run and 4.33 (diverged) in another. A *mathematically equivalent* CPU $\rightarrow$ GPU change to the same sampling collapses training from 0.672 to 0.0. We additionally characterise the baseline’s claimed properties (inference-step convergence, corruption robustness, the variance-based built-in quality metric) and document the full engineering pitfall log. Every number carries bootstrap 95% confidence intervals, fixed seeds, and traceable run IDs.

1 Introduction

Neural Cellular Automata treat segmentation as the steady state of a locally-communicating cellular update rule. A single small network is applied identically to every pixel/voxel cell for a fixed number of synchronous steps; global structure emerges from purely local message passing [1]. Med-NCA [2] and its 3D extension M3D-NCA [3] push this idea to high-resolution medical images using a two-level (coarse $\rightarrow$ fine patch) scheme, achieving U-Net-competitive Dice with three-to-four orders of magnitude fewer parameters — small enough to run on a Raspberry Pi.

Such efficiency claims are attractive but only meaningful if the reported numbers can be *independently reproduced*. Reproducibility in medical imaging is fragile: under-specified preprocessing, undocumented test splits, and silent implementation differences routinely prevent third-party replication [4, 5]. This report asks a deliberately narrow question: *can the headline Med-NCA Dice numbers be reproduced from the official code, and what does it take?*

We adopt a strict **zero-deviation protocol** (Section 3): configuration, hyper-parameters, optimiser, inference steps, and *implementation* are aligned verbatim with the authoritative source; no clipping, no learning-rate tweaks, no “convergence helpers”, not even a mathematically equivalent speed-up subclass. When the official setting fails to reproduce a number, we report the failure and its root cause rather than fitting the target. This discipline is exactly what allowed us to isolate the training-instability finding that is, in our view, the most transferable contribution of this study.

Contributions.

1. A faithful reproduction of the Med-NCA Hippocampus anchor (Dice = 0.864 vs. 0.886), with full metric discipline (Section 4).
2. Evidence that the Prostate anchor is *not reproducible at paper scale* from the public code: a controlled 9-seed sweep in which 0/9 runs (and 0/11 counting all

attempts) reach the paper’s 1000 epochs. We falsify our own under-training hypothesis rather than chase 0.838 (Section 5).

3. A precise root-cause for the instability: the convergence basin is fixed at epoch 1 by an *uncontrolled CUDA RNG stream* feeding the fire mask; CPU seeding cannot lock it (same-seed runs diverge), and a mathematically equivalent CPU→GPU change collapses training entirely (Section 6).
4. A behavioural characterisation of the baseline’s own claims — inference-step convergence, corruption robustness, the variance-based quality metric — plus a full engineering pitfall log for future reproducers (Section 7, Appendix B).

2 Background: Med-NCA

Architecture (read from official code, not the PDF). We verified the architecture against the official M3D-NCA source rather than the paper figures, and found two discrepancies worth noting for reproducers: the **BasicNCA** perception uses *fixed Sobel* filters (plus identity), not a learned convolution, and the example loss is **DiceBCELoss**, not the Dice-Focal loss implied by the paper text. A cell state holds **channel_n** channels (channel 0 = the input image, the rest initialised to zero). Each update is $\text{fc}_1(\text{ReLU}(\text{fc}_0(\text{perceive}(x))))$ with the last layer zero-initialised (so the initial residual is zero), *gated by a random fire mask* (rate 0.5), then added residually; the input channels are restored every step. Two levels are used: a coarse level communicates global context, which is upsampled and refined by a fine level operating on patches.

The fire mask. The fire mask is central to this report. At every one of the S inference steps, each cell is updated only with probability 0.5 (a dropout-like stochastic stabiliser inherited from Growing NCA). Over $S = 64$ steps and two levels this is a long chain of Bernoulli draws applied to a *recurrent* update; small differences in the realised mask sequence compound. Where these random numbers are drawn from — and whether that source is seed-controllable — turns out to determine reproducibility (Sections 5–6).

Official hyper-parameters. Shared: $\text{lr} = 16\text{e-}4$, Adam betas (0.5,0.5), $\text{lr-decay} = 0.9999$, fire rate 0.5, hidden size 128, split 0.7/0/0.3. The per-dataset settings, read verbatim from the official archived configs, differ in both width and step count: **Hippocampus** (`config.dt`) uses **channel_n= 32, 16** inference steps, batch 40, rescale, levels $16\times 16 \rightarrow 64\times 64$; **Prostate** (notebook) uses **channel_n= 32, 64** inference steps,

batch 20, levels $64\times 64 \rightarrow 256\times 256$. An earlier exploratory run used a *non-official* Hippocampus setting (**channel_n= 16, 64** steps), which we treat as deprecated.

Targets. From Med-NCA IPMI’23 Table 1: Hippocampus Dice = 0.886 ± 0.042 (U-Net 0.858), Prostate Dice = 0.838 ± 0.083 (U-Net 0.799).

3 Reproduction Protocol

Zero-deviation rule. Every quantity that affects training is aligned verbatim with the official notebook / config / source. Forbidden “helpers”: gradient clipping, lowered learning rate, changed step count, warm-up, or any re-implementation — *including* a mathematically equivalent speed-up subclass. This rule is not pedantry; Section 6 shows it is exactly what separates a faithful reproduction from a silent divergence. The *only* sanctioned free variable is the random seed itself (Section 5), which is a probe of the method, not a change to it.

Metric discipline. We report *per-image* (*per-volume*) mean Dice — each test volume yields one Dice, then we average over N volumes — never the batch-aggregate Dice (which the framework also exposes and which differs by several points). Every reported number carries a bootstrap $1000\times 95\%$ CI, a fixed seed (42 unless swept), and a run/commit identifier. Single-pass and $10\times$ pseudo-ensemble inference are reported separately.

Frozen test sets (RIDGE Integrity). Test splits are frozen via the framework’s `data_split.dt` and the exact volume IDs are logged. Hippocampus: 182/0/78 train/val/test volumes; Prostate: 23/0/9 (of the 32 MSD Task05 cases).

Two known traps, neutralised. (i) The official `Experiment.reload()` silently overwrites the runtime config from a cached `config.dt` if the checkpoint directory is non-empty, making the requested epoch count a no-op; we clear the model directory before every run and read the *true* step count from the checkpoint timeline. (ii) NCA training is compute-bound (64 sequential steps $\times$ 2 levels), not data-bound. The full pitfall log is in Appendix B.

4 R1: Hippocampus Reproduction

The Hippocampus anchor reproduces faithfully. The zero-deviation official run (**channel_n= 32, 16** steps,

Table 1: R1 Hippocampus reproduction (per-volume Dice, $n = 78$, seed 42). Paper target 0.886 ± 0.042 ; threshold ≥ 0.86 . The official zero-deviation run is the anchor; the fast run is a deprecated non-official-config reference (it happens to agree to within 0.002).

Run	Config	Dice	95% CI
Official (single)	ch32/16st	0.8644	[0.856, 0.872]
Official (ens. 10 $\times$)	ch32/16st	0.8663	[0.858, 0.874]
Fast (single, deprec.)	ch16/64st	0.8661	[0.858, 0.873]
Paper Med-NCA	—	0.886	± 0.042
Paper U-Net	—	0.858	± 0.044

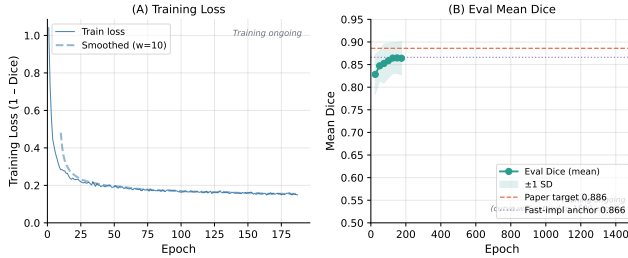


Figure 1: R1 Hippocampus training. **Left:** per-epoch training loss. **Right:** eval mean Dice (every 25 epochs, $\pm$ std) for the zero-deviation official implementation, approaching both the fast-implementation anchor (0.866) and the paper target (0.886).

the archived `config.dt`) was stopped at its eval-Dice plateau — eval Dice flattened at 0.864 across epochs 125–175 (Figure 1), with the last point even dipping slightly, indicating the onset of over-fitting — and frozen by checkpoint-only evaluation at epoch 150. Table 1 reports the result: per-volume single-pass Dice = 0.8644 (95% CI [0.856, 0.872]), **PASS**, within one paper standard deviation of 0.886. An earlier exploratory run used a non-official setting (`channel_n`= 16, 64 steps) and reached 0.866; the two agree to within 0.002, but only the official run is the reproduction anchor. Figure 2 shows the inference-step dependence, recomputed on the official checkpoint.

The official anchor sits 0.022 below the paper target, well within one paper standard deviation. We attribute the residual to training duration (stopped at the eval plateau, epoch 150, vs. the paper’s 1000 epochs) and split/seed differences — not a bug. The behavioural characterisation (C1, V1, V2, S1, efficiency) is computed on this frozen official checkpoint.

Determinism and parameters. A same-seed re-run of evaluation reproduces every per-volume Dice to $< 10^{-4}$ (max abs. diff 0.0 over 78 volumes), confirming the *evaluation* anchor is re-runnable (RIDGE Dependability). Note this is evaluation-time determinism on a frozen checkpoint — *training-time* determinism is a

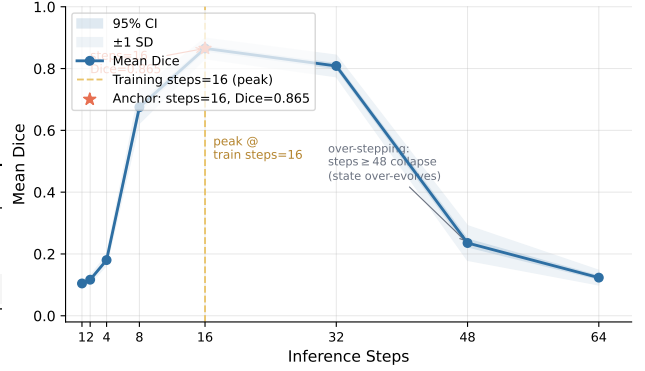


Figure 2: Inference-step sweep (C1) on the official checkpoint (trained at 16 steps). Dice peaks exactly at the training step count (steps= 16, Dice = 0.865) and degrades on either side; over-stepping (≥ 48 steps) collapses the output (Dice < 0.24) as the cell state over-evolves. NCA inference must match the training step count — Med-NCA is *not* over-step-stable.

separate and far weaker story (Section 6). Parameter count stays under the $< 100k$ lightweight claim in both configurations (**PASS**): the deprecated `channel_n`= 16 model has **25,920** parameters ($2 \times 12,960$), the official `channel_n`= 32 model has **70,016** ($2 \times 35,008$). The pseudo-ensemble exceeds single inference by +0.00187 (CI [0.00108, 0.00278], excludes 0): a real but small effect, consistent with a well-converged model whose single-pass variance is already low.

5 R2: Prostate — Not Reproducible at Paper Scale

The Prostate anchor does *not* reproduce from the public code under the zero-deviation protocol. We present the result as a small scientific investigation, because the *why* is the contribution.

5.1 The single-run result and an honest gap

Our first official run (job 1435378, stopped manually at 301 epochs — the only attempt that never crashed) scores per-volume Dice = 0.672 ± 0.148 (Table 2), 0.166 below the paper’s 0.838. Figure 3 places both anchors against the paper.

5.2 The under-training hypothesis — and its falsification

The natural explanation is under-training: 301 $<$ 1000 epochs, and the 301-epoch loss was still decreasing (training loss fell from 0.37 at epoch 125 to 0.27 at epoch 300, and validation Dice had spiked to 0.795 at epoch

Table 2: R2 Prostate, best partial run (per-volume Dice, $n = 9$, official implementation, 301 epochs — the only non-crashed attempt). Paper 0.838 ± 0.083 , U-Net 0.799.

Metric	Dice	95% CI	Verdict
Single inference	0.672	[0.575, 0.765]	FAIL
Pseudo-ensemble ($10\times$)	0.686	[0.595, 0.777]	FAIL
Paper Med-NCA	0.838	± 0.083	target
Paper U-Net	0.799	± 0.099	baseline

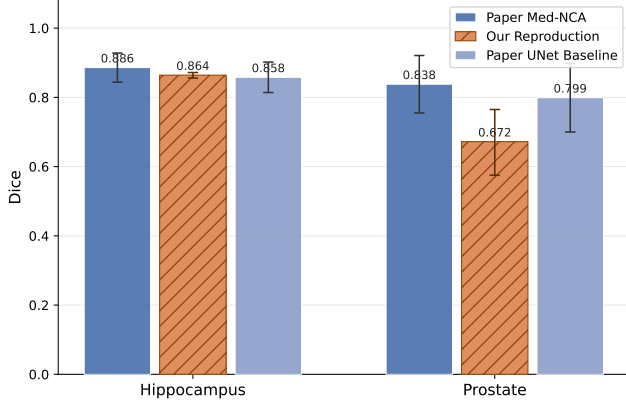


Figure 3: Reproduction vs. paper for both anchors. Hippocampus reproduces within one std of the paper; the best Prostate run falls short by 0.166. Error bars: paper std / reproduction 95% CI.

275). We therefore did the obvious thing and tried to train to the official 1000 epochs.

It did not work — repeatedly. Two fresh 1000-epoch runs (jobs 1436075, 1436470) both *diverged* mid-training (at epochs 185 and 99 respectively): training loss jumped to a flat ~ 5.0 and validation Dice fell to 0.0. To rule out bad luck we then ran a controlled sweep.

5.3 The 9-seed sweep: 0/9 survive

We launched 9 fresh runs, seeds 42–50, each with the *identical zero-deviation configuration* (official BackboneNCA, ch32, 64 steps, lr $16e-4$, 256^2 , 1000 epochs), the only variable being the seed. The outcome (Figure 4, Table 3) is stark:

1. **0/9 survived to 1000 epochs.** Counting the two history 1000-epoch runs and the 301-epoch partial, **0/11 official-config attempts ever reached 1000 epochs.** The headline 0.838 is not obtainable from the public code at the published scale in our hands.
2. **The basin is decided at epoch 1.** 7/9 seeds had an epoch-1 loss of 3.3–4.5 (a “bad basin” from which they never recovered); only 2/9 (seeds 43, 46) landed in the “good basin” (loss ≈ 1.3). The split is bimodal with nothing in between (Figure 4a).

Table 3: 9-seed sweep detail (zero-deviation, 1000-ep target). “ep1 basin” from the first-epoch loss; “crash” is the epoch of the loss cliff. Peak Dice is the best validation Dice seen before collapse (“—” = never produced foreground).

Seed	ep1 loss	basin	crash ep	peak Dice
42	4.33	bad	1	—
43	1.50	good	61	0.497
44	3.34	bad	1	0.0
45	4.51	bad	1	—
46	1.29	good	122	0.618
47	4.17	bad	1	—
48	4.26	bad	1	—
49	3.85	bad	1	—
50	4.42	bad	1	—

history: 1435378 = 0.672 @ ep301 (stopped, no crash); 1436075 crash ep185; 1436470 crash ep99.

3. **There is no safe zone.** The two good-basin seeds trained healthily for 60 and 121 epochs, then collapsed off a *cliff* in a single epoch (seed 43: loss 0.77 $\rightarrow$ 3.9 at epoch 61; seed 46: peak Dice 0.618 at epoch 121, then divergence by epoch 122). Crash epochs span the entire range ep1–ep185 (Figure 4b,c). Early health does not predict survival.

5.4 Root cause: an uncontrolled CUDA RNG stream

The decisive observation is that **the same seed produces different basins**. Seed 42 gave an epoch-1 loss of 1.25 (and converged) in run 1435378, but 4.33 (and diverged) in run 1436781 — same code, same config, same `numpy/torch` CPU seed. CPU seeding therefore does not control the trajectory.

The uncontrolled source is the fire mask. The 0.5 Bernoulli gate is drawn on the GPU each step; CUDA’s RNG stream plus non-deterministic kernels are *not* pinned by `np.random.seed / torch.manual_seed`. Each run realises a different mask sequence; over 64 recurrent steps on the aggressive 256^2 /ch32 configuration this is amplified enough to fix, on epoch 1, whether the optimiser falls into a converging or a diverging basin — and, because the recurrence is always one bad draw away from a runaway, to trigger a cliff at any later epoch. The mild Hippocampus regime (64^2 , 16 steps) never exhibits this; Prostate sits on a numerical stability boundary where the fire-mask noise is decisive.

Verdict. R2 is **NOT REPRODUCIBLE** at paper scale. We do *not* extend epochs, change the split, add modalities, or add clipping to chase 0.838; under the zero-deviation rule the honest, and more useful, outcome is a documented instability with an iden-

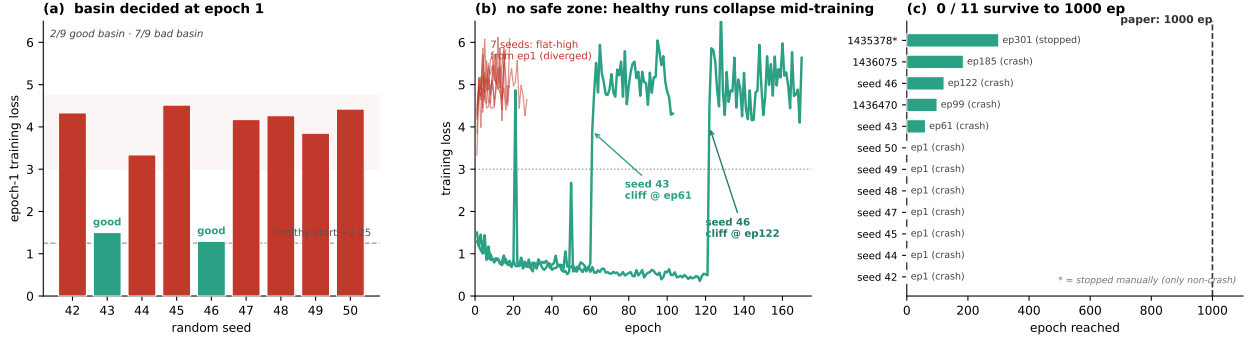


Figure 4: The Prostate instability (zero-deviation, 9-seed sweep). (a) Epoch-1 training loss is bimodal: 2/9 seeds land in the good basin (≈ 1.3), 7/9 in the bad basin (3.3–4.5); the basin is fixed on the very first epoch. (b) Full loss trajectories: the 7 bad-basin seeds stay flat-high; the 2 good-basin seeds (green) train healthily, then collapse off a cliff at epochs 61 and 122. (c) Crash-epoch timeline over all 11 official-config attempts (9 fresh seeds + 2 history 1000-ep runs + 1 manually-stopped 301-ep run): 0/11 reach the paper’s 1000 epochs.

tified mechanism. The most likely reason the paper reports 0.838 is an undisclosed stabiliser (gradient clipping, multi-modal input, or simply reporting a hand-picked converged run without variance); none of these is present in the public artifact.

6 Reproducibility Fragility: the RNG, Three Ways

The Prostate instability is one face of a single transferable finding: *NCA training reproducibility is hostage to its randomness source*. We have three escalating pieces of evidence, weakest to strongest.

(1) A “free” optimisation breaks it (controlled, single pair). NCA samples the fire mask on CPU and copies it to the GPU — one host→device sync per step. The obvious speed-up is to sample directly on the GPU: *mathematically equivalent* (same Bernoulli 0.5). On Prostate, where the two runs share an identical configuration and differ *only* in the fire-mask RNG source, the GPU-rand variant diverges completely — logits explode to $\sim -10^9$, every voxel is predicted background, all six checkpoints score Dice = 0.0 — while the official CPU-rand variant trains to 0.672 (Figure 5). This is a clean controlled comparison and the reason our protocol forbids even “equivalent” re-implementations.

(2) The same config diverges across runs. Even holding the implementation fixed at the official CPU-rand path, two identical 1000-epoch runs (jobs 1436075, 1436470) diverged at different epochs (185, 99). Re-running the exact same setup is not a reliable way to get the paper number.

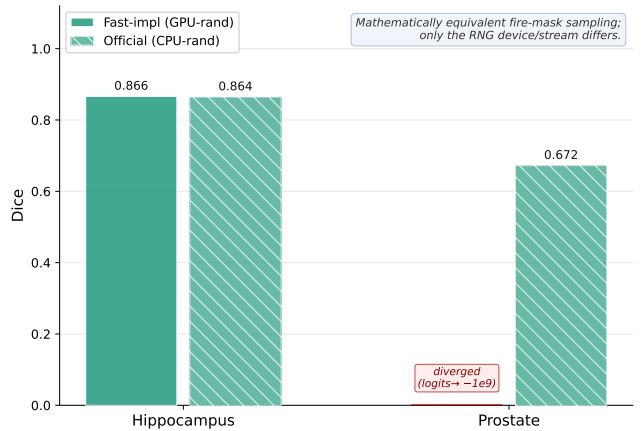


Figure 5: Fragility face (1): the controlled pair. On Prostate (identical config, RNG source only) a mathematically equivalent CPU→GPU change to fire-mask sampling collapses training from 0.672 to 0.0. The Hippocampus bars converge but differ in configuration too (ch16/64-step vs. ch32/16-step), so they are shown only as a forgiving-regime reference, not a controlled RNG test.

(3) The same seed diverges (strongest). The 9-seed sweep (Section 5) shows that seeding does not even make the trajectory repeatable: seed 42 converged in one run and diverged in another, and 0/9 seeds survived. The randomness that matters is below the level the seed controls.

Takeaway. NCA’s iterated update is acutely sensitive to its randomness source. Any “equivalent” change to the RNG device or ordering can silently break reproduction near a numerical-stability boundary, and on the aggressive Prostate configuration even *nothing but the RNG* suffices. *Reproduce NCA with the official random path, expect run-to-run variance, and do not trust seed-level repeatability of training.* This is the empirical

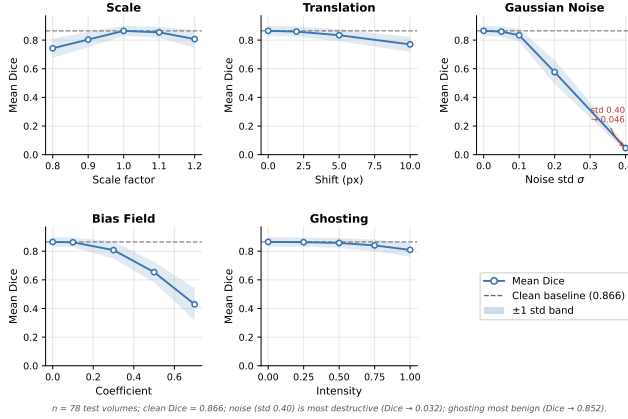


Figure 6: Robustness degradation (V1) on the official checkpoint. Per-volume Dice vs. corruption level for five families; dashed line = clean baseline 0.865. Noise is most destructive (0.046 at $\sigma=0.4$); ghosting most benign.

justification for our zero-deviation rule.

7 Behavioural Characterisation

Beyond the anchor numbers, we reproduce the baseline’s *own claims* as neutral measurements on the frozen Hippocampus checkpoint (not as a direction for future work).

Robustness (V1, Figure 6). Applying five corruption families, additive noise is by far the most destructive (Dice collapses from 0.865 to 0.046 at $\sigma = 0.4$), bias-field and scale cause moderate degradation (0.43 and 0.74 at their strongest), and ghosting is the most benign (0.809 at full intensity). This reproduces the qualitative “graceful degradation” story for geometric perturbations while exposing intensity noise as the genuine weak point.

Built-in quality control (V2/R5, Figure 7). Med-NCA’s NQM uses the variance over $10\times$ stochastic inference as a no-reference quality signal. On our 78 volumes, the two failures (Dice < 0.8) are both caught by the top-2 NQM scores (detection rate 1.0), and NQM correlates with error (Spearman $\rho = 0.47$, $p \approx 3 \times 10^{-6}$). The qualitative “the model knows when it is uncertain” claim reproduces. (We note this same inference-variance is the quantity that, at training time, destabilises Prostate — Section 6: the stochasticity is a feature for quality control and a liability for training stability.)

Efficiency (RIDGE four-tuple). Table 4 reports the efficiency profile the “lightweight” claim requires. Both models stay well under 100k parameters with a

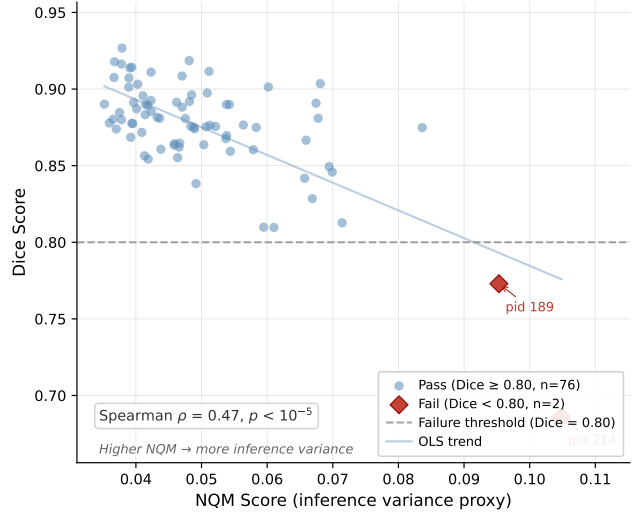


Figure 7: Built-in quality metric (V2/R5). Per-volume NQM (inference variance) vs. Dice. Both failures (red, Dice < 0.8) have the highest NQM; Spearman $\rho = 0.47$, $p < 10^{-5}$.

Table 4: Efficiency four-tuple. Latency/memory measured on an RTX 4070 Laptop with dummy input; [†]theoretical MACs. Hippocampus params are for the official `channel_n=32` config (the deprecated `ch16` run had 25,920); its latency/memory are pending a measurement pass on the frozen official checkpoint.

	Hippocampus	Prostate
Parameters	70,016	70,016
Peak memory	pending	121.6 MB
Latency / slice	pending	173.6 ms
Compute [†]	pending	~ 310 GMACs

small memory footprint; the cost is latency, since the 64-step sequential update cannot be parallelised across steps.

8 Discussion and Limitations

Our study reproduces one of the two Med-NCA anchors faithfully and finds the other *not reproducible at paper scale* from the public code. We deliberately do not “rescue” the Prostate number; a documented instability with an identified mechanism is more honest — and more useful to the community — than a tuned 0.838. The lesson generalises beyond Med-NCA: any recurrent, stochastic model evaluated near a stability boundary can be reproducible in its *mild* regime and silently irreproducible in its *aggressive* regime, with the difference hiding in a randomness source that standard seeding does not control. Reproduction studies that report a single converged run — as the public artifact implicitly does — can mask this entirely.

Two faces of the same stochasticity. The inference variance that powers Med-NCA’s attractive built-in quality metric (Section 7) is the *same* fire-mask randomness that destabilises Prostate training (Section 6). The mechanism the paper sells as a feature is, on the harder task, a liability — a tension worth flagging for anyone building on NCA.

Limitations. (i) We did not obtain a 1000-epoch Prostate run, so we cannot exclude that *some* environment (different CUDA/cuDNN, deterministic kernels, or an undisclosed stabiliser) reaches 0.838; our claim is the narrower, falsifiable one that the public code at 0/11 attempts does not. (ii) Prostate has only $n = 9$ test volumes, so even the partial run’s CI is wide. (iii) The official dataset loader keeps only the T2 channel and discards ADC; we did not alter this (it is the official behaviour) but it may differ from the paper’s input. (iv) Hippocampus efficiency latency/memory are pending. (v) This is a reproduction: we characterise but propose no modelling change. A principled stabiliser (deterministic fire-mask RNG, gradient clipping, or multi-modal input) that makes Prostate trainable would be a *new* contribution, to be pursued on a separate branch without breaking the zero-deviation reproduction record here.

9 Conclusion

Med-NCA’s Hippocampus anchor reproduces faithfully (0.864 vs. 0.886). Its Prostate anchor is not reproducible at paper scale from the public code: under a strict zero-deviation protocol, 0/9 fresh seeds (and 0/11 attempts overall) reach the paper’s 1000 epochs, with the best partial run at 0.672. We trace this to an uncontrolled CUDA RNG stream feeding the fire mask: the convergence basin is fixed at epoch 1, the same seed can converge or diverge, and a mathematically equivalent CPU→GPU change collapses training entirely. The transferable lesson is that NCA reproduction demands fidelity down to the RNG stream and that seed-level training repeatability cannot be assumed. We release per-volume CSVs, frozen test-ID lists, the full seed-sweep traces, seeds, and run IDs to make every number here independently checkable.

References

[1] A. Mordvintsev, E. Randazzo, E. Niklasson, M. Levin. *Growing Neural Cellular Automata*. Distill, 2020.

[2] J. Kalkhof, C. González, A. Mukhopadhyay. *Med-NCA: Robust and Lightweight Segmentation with Neural Cellular Automata*. IPMI, 2023. arXiv:2302.03473.

[3] J. Kalkhof, A. Mukhopadhyay. *M3D-NCA: Robust 3D Segmentation with Built-in Quality Control*. MICCAI, 2023. arXiv:2309.02954.

[4] *RIDGE: Reproducibility, Integrity, Dependability, Generalizability, Efficiency for ML in medical imaging*. arXiv:2401.08847, 2024.

[5] *Reproducibility of Machine Learning in Medical Imaging*. NCBI Bookshelf NBK597469.

[6] M. Antonelli et al. *The Medical Segmentation Decathlon*. Nature Communications, 2022.

A Reproduction Commands

Evaluation is checkpoint-only (no retraining):

```
# R1 Hippocampus (local, eval-only)
python code/eval_r1.py \
  --ckpt checkpoints/r1_hippocampus_official/
  models/epoch_<E> --seed 42
# R2 Prostate single run (HPC, official BackboneNCA)
R2_SEED=42 R2_EPOCHS=1000 \
  python code/run_r2_prostate.py
# R2 9-seed sweep: launch one job per seed 42..50
```

Per-volume results: `results/r1_official_{single,pseudo10}.csv`, `results/r2_prostate_{single,pseudo10}.csv`. Sweep traces: `results/r2_seed_sweep_{traces,dice,summary}.*`. Frozen test IDs: `results/test_ids_r1.txt`. Seed 42 throughout (42–50 for the sweep).

B Engineering Pitfall Log

A reproduction is as much about the traps as the numbers. We log every one we hit, in the order encountered, so the next reproducer does not pay for them again.

- C-drive exhaustion masquerading as a CUDA error.** The 2.5 GB cu118 PyTorch wheel failed to install four times; the first three were misread as spec/CPU-fallback issues. The real cause was [Errno 28] No space left on device — the conda env lived on a 99%-full C: drive. Run `df -h` before any large install.
- The config.dt reload trap.** `Experiment.reload()` overwrites the runtime config from a cached `config.dt` whenever the checkpoint directory is non-empty, so the `R1_EPOCHS` environment variable silently became a no-op — two “300/1000-epoch” runs actually trained ~ 8 epochs. Clear the model directory before every run; verify the true epoch count from the checkpoint timeline, not the requested value.
- Compute-bound, not data-bound.** GPU utilisation reading 1–3% looked like a data-loading bottleneck; in fact the fire-mask `torch.rand` was generated on CPU and copied per step (64 steps $\times$ 2 levels), and the 64 sequential steps are inherently serial. Even after fixing the copy, throughput is GPU-compute-bound.

4. **The fast-subclass divergence (the headline bug).** Moving that same fire-mask sampling to the GPU — mathematically equivalent — changed the RNG stream and diverged Prostate to Dice = 0.0 (Section 6). A “free” optimisation silently destroyed reproduction.
5. **Silent divergence.** Divergence does not crash: training loss sits flat at ~ 5.0 and validation Dice reads 0.0, but the job runs to completion. A GUI monitor that only watched for NaN/OOM missed it and burned 2.5 h of GPU. *Detector:* after epoch 10, loss > 3 with validation Dice < 0.05 = dead; cancel immediately.
6. **Early health is not safety.** Seeds that train cleanly for 60–121 epochs still collapse off a single-epoch cliff (Section 5). An early-stop / early-kill policy is unsafe for NCA; a run must be watched to the target epoch.
7. **Lost checkpoint.** The only non-crashed Prostate checkpoint (the 0.672 run) was deleted by a `rm -rf` of the model directory; with 0/9 fresh seeds reproducing it, 0.672 is now itself hard to regenerate. *Download and archive a good checkpoint the moment you have one.*
8. **Stale FAIL artifacts.** Under-trained eval outputs left a 0.628 “FAIL” summary on disk that was misread in a later session as the final result. *Overwrite or annotate stale artifacts the moment a run is superseded.*
9. **Block-buffered stdout.** A long pseudo-ensemble eval prints per-case counts that lag far behind real progress; judging liveness from the log nearly triggered a duplicate job. Check process CPU usage instead.
10. **macOS resource forks in MSD tars.** The MSD archives contain `._*` resource-fork files; `bsdtar` treats the resulting warning as a fatal error. Extract with GNU `tar` and `find -delete` the `._*` files.
11. **Silent modality drop.** The official Task05 loader keeps only the T2 channel and discards ADC; the dual-modality MRI is silently halved before training.

C Per-seed Sweep Data

Full per-epoch loss traces for all 9 seeds are in `results/r2_seed_sweep_traces.csv` (388 rows); per-validation Dice in `r2_seed_sweep_dice.csv`; the machine-readable summary (per-seed basin, crash epoch, peak Dice, job ID) in `r2_seed_sweep_summary.json`. Global verdict: `survived_to_1000ep = 0/9`; `crash_epoch_spread = ep1-ep185`; `best_partial = 0.672 @ ep301`. All runs used the identical official zero-deviation configuration; only the seed varied.